In [1]:
```
# Exp :- 10
```

In [2]:
```
# Aim : To perform Operation Simple Linear Regression for Salary Data(SLR
```

In [1]:
```
#Name:Anurag Jagnade
#Roll:27
#sec:C
#subject:DSS
#Date:
```

In [2]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

In [3]:
```
import os
```

In [4]:
```
os.getcwd()
```

Out[4]:
```
'C:\\Users\\Anurag'
```

In [6]:
```
os.chdir("C:\\Users\\Anurag\\Desktop")
```

In [7]:
```
data=pd.read_csv("Salary.csv")
```

In [8]:
```
data.head()
```

Out[8]:

	YearsExperience	Salary
0	1.1	39343
1	1.3	46205
2	1.5	37731
3	2.0	43525
4	2.2	39891

In [9]:
```
data.tail()
```

Out[9]:

	YearsExperience	Salary
30	11.2	127345
31	11.5	126756
32	12.3	128765
33	12.9	135675
34	13.5	139465

In [78…
```
data.head(30)
```

Out[78]:

	YearsExperience	Salary
0	1.1	39343
1	1.3	46205
2	1.5	37731
3	2.0	43525
4	2.2	39891
5	2.9	56642
6	3.0	60150
7	3.2	54445
8	3.2	64445
9	3.7	57189
10	3.9	63218
11	4.0	55794
12	4.0	56957
13	4.1	57081
14	4.5	61111
15	4.9	67938
16	5.1	66029
17	5.3	83088
18	5.9	81363
19	6.0	93940
20	6.8	91738
21	7.1	98273
22	7.9	101302
23	8.2	113812
24	8.7	109431
25	9.0	105582
26	9.5	116969
27	9.6	112635
28	10.3	122391
29	10.5	121872

In [10…
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 35 entries, 0 to 34
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  35 non-null     float64
 1   Salary           35 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 688.0 bytes
```

In [11…]
```
data.describe()
```

Out[11]:

	YearsExperience	Salary
count	35.000000	35.000000
mean	6.308571	83945.600000
std	3.618610	32162.673003
min	1.100000	37731.000000
25%	3.450000	57019.000000
50%	5.300000	81363.000000
75%	9.250000	113223.500000
max	13.500000	139465.000000

In [12…]
```
data.shape
```

Out[12]:
```
(35, 2)
```

In [13…]
```
data.size
```

Out[13]:
```
70
```

In [14…]
```
data.ndim
```

Out[14]:
```
2
```

In [15…]
```
data.isnull().sum()
```

Out[15]:
```
YearsExperience    0
Salary             0
dtype: int64
```

In [16…]
```
#Independent and Dependent Variables
#x = data['YearsExperience']
#y = data['Salary']
```

Independent and Dependent Variables

In [17…
```
X=data.drop('Salary',axis=1)
```

In [18…
```
X.head()
```

Out[18]:

	YearsExperience
0	1.1
1	1.3
2	1.5
3	2.0
4	2.2

In [19…
```
y=data.Salary
```

In [20…
```
y.head()
```

Out[20]:
```
0    39343
1    46205
2    37731
3    43525
4    39891
Name: Salary, dtype: int64
```

In [21…
```
from sklearn.model_selection import train_test_split
```

In [22…
```
X_train,X_test,y_train,y_test=train_test_split(X,y,random_state=0,test_si
```

In [23…
```
print(X_train.shape)
```
```
(24, 1)
```

In [24…
```
X_test.shape
```

Out[24]:
```
(11, 1)
```

In [ ]:
```

```

Model Fitting

In [25…
```
from sklearn.linear_model import LinearRegression
```

```
LR=LinearRegression()
LR.fit(X_train,y_train)
```

Out[25]:
```
LinearRegression()
```

In [26…
```
#Assigning Coefficient (slope) to m
m = LR.coef_
```

In [27…
```
print("Coefficient  :" , m)
```

```
Coefficient  : [8528.00205699]
```

In [28…
```
#Assigning Y-intercept to a
c = LR.intercept_
```

In [29…
```
print("Intercept : ", c)
```

```
Intercept :  30653.80404187284
```

y=mx+c

Prediction

In [30…
```
y_pred=LR.predict(X_test)
```

In [31…
```
y_pred
```

Out[31]:
```
array([120197.8256403 ,  88644.21802942,  74146.61453254, 118492.225228
9 ,
        98025.02029212,  72441.01412114,  63913.01206415,  43445.807127
36,
        64765.81226984, 112522.623789  , 107405.82255481])
```

In [32…
```
y_test
```

Out[32]:
```
29    121872
20     91738
16     66029
28    122391
22    101302
15     67938
10     63218
2      37731
11     55794
27    112635
25    105582
Name: Salary, dtype: int64
```

Evaluation metrics

In [33…
```
from sklearn import metrics
```

Accuracy

In [34…
```
Accuracy = LR.score(X_test, y_test)
Accuracy
```

Out[34]: 0.9714499470616078

MAE

In [35…
```
print(metrics.mean_absolute_error(y_test,y_pred))
```

3807.469971826513

MSE

In [36…
```
print(np.sqrt(metrics.mean_squared_error(y_test,y_pred)))
```

4685.185783311836

RMSE

In [37…
```
rmse = np.sqrt(metrics.mean_squared_error(y_test,y_pred))
print("RMSE:", rmse)
```

RMSE: 4685.185783311836

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: